סיכום תרגולים 2027 – שפות תכנות

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד. שימו לב כי בסיכום זה התרגולים התכנותיים בלבד. אלו התאורטיים יותר נמצאים בקובץ אחר.

תוכן עניינים:

תרגול 1: prolog 2

תרגול 2: PROLOG BACKTRACKING 9

תרגול 4: SML 15

תרגול 1

שפות תכנות מחולקות לשני סוגים גדולים:

1. **שפות הצהרתיות (declarative):** בשפות אלו המיקוד הוא על מה המחשב צריך לעשות. השפה עוסקת בהצהרה על אובייקטים. למשל: CSS,HTML,SQL.

גם שפות הצהרתיות מתחלקות לשני סוגים:

- שפות בהן פונקציות קוראות לפונקציות – דוגמת Ocmal.
- Logic: כאן ישנם חוקים לוגיים וקשרים. למשל – Prolog עליה נדבר היום.

2. שפות מצוות (imperative): המיקוד כאן הוא איך המחשב צריך לעשות. השפה עוסקת בפקודות למערכת, למשל Java,python,C,C++ וכו'.

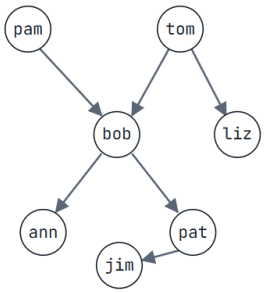
שפות מצוות מתחלקות לשלושה חלקים:

- **Von Neumann:** המודל החישובי של ון ניומן, מודל חישובי של שינוי משתנים.
- **Object-Oriented**: מודל מובנה של חישוב אובייקטים והאינטראקציה בניהם.
- **Scripting languages:** שפות סקריפטים, תיאום בין רכיבים.

כלומר, בC אנו כותבים אלגוריתם כמו לולאה, תנאי והשמה. בפרולוג נכתוב ידע – עובדות: "טום הוא ההורה של בוב" וחוקים: " סבא הוא הורה של הורה". ואז – נשאל שאלות. מנוע הפרולוג מחפש בעצמו את הדרך להוכיח את התשובה.

Prolog

על מנת להבין כיצד מתכנתים בפרולוג, נתבונן בדוגמה של עץ משפחה:



בשלב הראשון, נרצה לקבוע עובדות: למשל, ליז היא הבת של טום.

```
?- assert(parent(pam, bob)).
true.
?- assert(parent(tom, bob)).
true.
?- assert(parent(tom, liz)).
true.
?- assert(parent(bob, ann)).
true.
?- assert(parent(bob, pat)).
true.
?- assert(parent(pt, jim)).
true.
```

ואז, ניתן לכתוב שאילתה. למשל – האם ליז היא ההורה של PAT?

והתשובה -

```
?- parent(bob, pat).
true.
?- parent(liz, pat).
false.
```

כמובן – parent איננה פקודה בפרולוג. זה שם שהמצאנו. פרולוג לא יודע מה זה הורה, יכולנו לקרוא לזה גם בננה. מה שאומרים לפרו-לוג, יש לנו קשר בשם parent בין בוב לpat. זה הכל.

Assert: פקודה מובנית בשפה, באמצעותה מוסיפים לזיכרון. פרולוג עונה true: "הוספתי את המידע".

לאחר שהעובדות כבר בזיכרון, ניתן לשאול באמצעות הוספת "?" בתחילת השורה משמאל ולקבל true or false.

- בפרולוג כל פקודה מסתיימת בנקודה. ללא הנקודה פרולוג לא מריץ את השאילתא.
- בפרולוג – אנחנו מגדירים פרדיקטים (יחסים) כעובדות.
- שאילתא (Query), בודקת אם היחס המבוקש נמצא בפרדיקט שהגדרנו.

הכנסת עובדות לבסיס הנתונים:

פרט לassert, ישנה דרך נוספת לגרום לסביבת העבודה להכיר פרויקט: באמצעות טעינה מקובץ. כותבים את כל העובדות בקובץ עם סיומת pl. וטוענים אותו עם consult(<name of file without .pl>)

לדוגמה:

file.pl

```
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).
```

```
?- consult(file).
true.

?- parent(bob, pat).
true.

?- parent(bob, or).
false.
```

- בסיס הנתונים בו שמורות כל העובדות נקרא logicbase.

פרדיקטים

פרדיקט מוגדר ע"י השם ומספר הארגומנטים שלו. למשל:

$$pred(arg1, arg2, \ldots, argN)$$

- שני פרדיקטים שונים עם אותו השם אך מס' ארגומנטים שונה – נחשבים לזוג פרדיקטים שונים.
- פרדיקט יכול להיות עם ארגומנט אחד בודד. זה משמש לרוב לציון עובדות בlogicbase.
- פרדיקט ללא ארגומנטים נכתב כך: .pred

ישנם מספר סוגי פרדיקטים:

סוג	מה זה	דוגמאות
מספר	שלם או ממשי	`28` `3.14`
אטום	קבוע: טקסט שמתחיל באות קטנה, או טקסט בין גרשיים `'…'`	`bob` `'Bar-Ilan'`
משתנה	מתחיל באות גדולה או ב-`_` (קו תחתון לבד = משתנה אנונימי)	`X` `Name` `_`
מבנה	נלמד בהמשך על סוגי מבנים בפרולוג	`point(1,2)`

- חשוב לזכור כי אות קטנה הוא ערך קבוע, ואות גדולה הוא משתנה שפרולוג ימלא עבורך.

פרדיקטים ופונקציות: בפרולוג משתמשים בפרדיקטים ופונקציות (שמבחינת מימוש נראות כמו פרדיקטים) בשביל לייצג עובדות בlogicbase, בשביל ייצוג של מבני נתונים וכן לייצג פונקציות לחישובים שונים.

- חשוב: בפרולוג אין return, פונקציה היא פרדיקט שאחד הארגומנטים שלה הוא **הפלט**. במקום len=size(list) נכתוב size(List, Len). הפרולוג ימלא את Len.

<u>דוגמה:</u>

```
parent(tom, bob).                  % עובדה
location(apple, kitchen).          % עובדה
costumer("Zvi", 28, 'Bar-Ilan').   % מבנה נתונים
size([1,2,3], N).                  % "פונקציה": N 3 יקבל
```

בדוגמה האחרונה – פרולוג רואה שN מתחיל באות גדולה לכן מבין שהוא משתנה שיש למלא. נשים לב שכרגע size לא הוגדרה לכן זה יחזיר שגיאה – בהמשך נטפל בזה.

פרדיקטים עם משתנים: ניתן לשלוח שאילתות עם משתנים על מנת לקבל מידע. למשל:

```
?- parent(X, liz).
X = tom.
```

```
?- parent(bob, X).
X = ann ;
X = pat.
```

logicbase

```
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).
```

נשים לב כי מקבלים רק את התשובה הראשונה, לפי הסדר שנכנס לlogicbase. לחיצה על נקודה פסיק תביא את התשובה הבאה – אם ישנה.

פרולוג עובר על העובדות לפי הסדר בקובץ ומחפש עובדה שניתן להתאים לשאילתה. ההתאמה הנ"ל נקרא **יוניפיקציה**. המשתנה X מקבל את הערך שגורם לשני הצדדים להיות זהים.

כמו כן, ניתן לשים משתנים בשני הארגומנטים. ושוב עם נקודה פסיק לקבל את כל המידע:

```
?- parent(X, Y).
X = pam,
Y = bob ;
X = tom,
Y = bob ;
X = tom,
Y = liz ;
```

- פרדיקט על משתנה: ניתן להגדיר פרדיקט על משתנה. המשמעות היא מעניינת – כל ארגומנט שיתקבל יקיים את הפרדיקט. נשים לב כי משתנה בעובדה הוא כמו $\forall x$ בלוגיקה. לכן כל שאילתא כאן בדוגמה מסוג sleep(bla-bla) מצליחה:

```
?- assert(sleep(X)).
true.

?- sleep(Zvi).
true.

?- sleep(students).
true.
```

שאילתות עם האופרטור וגם: ע"י הוספת פסיק ניתן לבצע שאילתות מהצורה וגם. למשל – בהינתן מאגר המידע שעליו אנחנו עובדים, שאילתא כמו:

```
?- parent(Y,jim), parent(X,Y).
Y = pat,
X = bob.
```

היא – מי הוא אבא של ג'ים (Y) וגם X שהוא אבא של Y? פרולוג הולך מחפש ומוצא: pat, bob. כיצד בפועל פרולוג פותר זאת? הוא מתחיל במטרה השמאלית – היא נותנת Y=pat. ואז המטרה השמאלית הופכת להיות $parent(Pat, X)$ שעבורה פרולוג מחזיר את BOB.

שאילתות עם האופרטור או: כאן יש להוסיף נקודה פסיק. למשל:

```
?- parent(X,jim); parent(X,ann).
X = pat ;
X = bob.
```

ביצענו את השאילתה $parent(X, jim)\ or\ parent(X, ann)$. כלומר – תביא לי את ההורה של ג'ים או אן. מבחינת הסדר שפרולוג מחזיר – קודם את תשובות הצד השמאלי ורק אז את תשובות הצד הימני בשאילתה.

הערות: ניתן לכתוב בפרולוג הערות, הערת שורה מתחילה בסימון % והערה ארוכה כך:

$$/*\ your\ comment\ \ldots.*/$$

- Write("Hello world") הוא פרדיקט שמדפיס.
- Nl – עושה זאת בשורה חדשה.
- Halt: עוצר את התוכנית.

```
?- write('hello world'), nl, halt.
hello world
zvi@com169433:~$
```

נשים לב שיש פסיקים בדוגמה - כי זה פשוט "וגם". פרולוג מריץ משמאל לימין – מדפיס, יורד שורה ויוצא.

מבנה: ניתן להשתמש בפונקציות כייצוג של מבנה אשר מחבר כמה פרמטרים יחד. למשל:

```
?- assert(costumer("Zvi", 28, Bar-Ilan)).
true.

?- costumer(X, 28, Bar-Ilan).
X = "Zvi".
```

כאן הגדרנו פונקציה באופן רגיל. "המבנה" הוא בצורת התפיסה שלנו. נשים לב כי שאלנו על X כמשתנה, קיבלנו צבי. ניתן לשאול ע"י השמת משתנה במקום אשר רוצים לגלות ולקבל מי מקיים את הארגומנט, שאיננו משתנה.

<u>דוגמה:</u> נתבונן על קובץ הpl הבא –

door.pl

```
room(kitchen).
room(office).
room(hall).

door(office, hall).
door(kitchen, office).

location(desk, office).
location(apple, kitchen).
location(computer, office).
```

```
?- room(X).
X = kitchen ;
X = office ;
X = hall.
```

הפרדיקט ROOM מגדיר כי ביחס 3 איברים. לכן ניתן לבצע שאילתא עם משתנה X ועם נקודה פסיק לקבל את כל מי שבחדר. פעולה זו נקראת **Binding:** קישור משתנה. המשתנה X נקשר לערך, בכל פעם שלוחצים נקודה פסיק פרולוג משחרר את הקישור הקודם ומנסה את העובדה הבא. הסדר חשוב בקישור!

- הבעיה: נשים לב כי הגדרנו door(office, hall) ולכן door(hall, office) יכשל. **אבל דלת היא סימטרית!** פרולוג לא יודע זאת כמובן. הפתרון הנאיבי הוא לכתוב כל דלת פעמיים. אבל יש פתרון טוב יותר: יש מעבר מX לY אם יש דלת מX לY או מY לX.

חוקים: עובדות נתנו לנו דרך לתאר יחסים ועובדות. למשל המשרד הוא חדר .room(office) מלבד עובדות, ניתן להגדיר בפרולוג חוקים – חוקים מאפשרים לכתוב משפטים מהסוג "אם X מתקיים, אז Y מתקיים".

התחביר	`head :- body.`
מתפרש כ-	`if body then head.`

כיצד קוראים את זה? את :- קוראים כמו חץ שמאלי ← הראש הוא המסקנה, והגוף הוא התנאי. הסדר הפוך מif רגיל. למשל:

$$grandfather(X,Z) :- parent(X,Y), parent(Y,Z)$$

אומר כי X סבא של Z אם קיים Y – הורה, כך שX הורה של Y וגם Y הורה של Z.

כעת כיצד מממשים דלת דו צדדית? בקלות:

```
connect(X,Y) :- door(X,Y).
connect(X,Y) :- door(Y,X).
```

המשמעות היא שהדרך להתחבר בין X לY היא אם קיימת דלת מX לY, ואפשרות נוסף אם קיימת מY לX.

חוקים – סמנטיקה

בהינתן החוק $P :- Q, R.$ ניתן לומר כי:

#	היגד	סוג
1	P אמת רק אם Q וגם R אמת.	**משמעות הצהרתית** `declarative meaning`
2	מ-Q ומ-R נובע P.	
3	כדי לפתור את הבעיה P, יש לפתור קודם את תת-הבעיה Q ואחר כך את תת-הבעיה R.	**משמעות פרוצדורלית** `procedural meaning`
4	כדי לספק את P, יש לספק קודם את Q ואחר כך את R.	

הסוג הצהרתית משמע מה נכון, לוגיקה טהורה. סדר איננו משנה. פרוצדורלית – כיצד פרולוג מחושב בפועל. כאן הסדר כן משנה: פרולוג תמיד מנסה משמאל לימין, ומלמעלה למטה. מדוע זה חשוב? שתי תוכניות יכולות להיות שקולות לוגית אך אחת תעבוד והשנייה תכנס ללולאה או תיתן שגיאה בגלל סדר המטרות.

אריתמטיקה:

```
?- X = 1 + 2.
X = 1+2.

?- X is 1 + 2.
X = 3.
```

לפרולוג הוסיפו את האפשרות לחשב ביטויים אריתמטיים. אך כיוון שהשפה עובדת באמצעות השוואת תבניות יש לכתוב במפורש שרוצים לחשב את הביטוי עם האופרטור is. האופרטורים הנתמכים הם -,+,*,/,mod. דוגמה:

Is מחשב את הצד הימני ומתאים את התוצאה לצד השמאלי. הצד הימני של is חייב להיות ידוע לגמרי, אחרת שגיאה. כמו כן פרולוג תומכת באופרטורים שמשווים בין שני ביטויים אריתמטיים:

אופרטור	משמעות
`X > Y`	גדול
`X < Y`	קטן
`X >= Y`	גדול או שווה
`X =< Y`	קטן או שווה (שימו לב לכתיב!)
`X =:= Y`	הערכים של X ו-Y זהים
`X =\= Y`	הערכים של X ו-Y שונים

אופרטורי השוואה מחשבים את שני הצדדים לפני ההשוואה! בניגוד לis למשל שמחשב רק את צד ימין. קטן או שווה נראה כמו מתמטיקה, ולא <=. זה נראה כמו חץ – והוא שמור.

- האופרטור = משווה בין מחרוזות. לעומת זאת, האופרטור X=:=Y מחשב את X ואת Y ומחזיר אם הם שווים.

```
?- 1 + 2 = 2 + 1.
false.
```

```
?- 1 + 2 =:= 2 + 1.
true.
```

<u>דוגמה:</u> פרדיקט שמחשב $gcd(a,b)$. המחלק המשותף הגדול ביותר של זוג מספרים , ידוע כי עבור a>b מתקיים $gcd(a,b) = gcd(a-b,b)$ ועל סמך הטענה הזו נציג אלגוריתם בפרולוג. למעשה, הגדרנו שלושה חוקים:

1. Gcd של X עם עצמו – הוא X.
2. אם X<Y אזי מחסירים את הקטן מהגדול, כלומר מחליפים את המשתנה Y להיות $Y1 = Y - X$.
3. אחרת, בדומה.

```
gcd.pl
```
```
gcd(X, X, X).
gcd(X, Y, D) :-
    X < Y,
    Y1 is Y - X,
    gcd(X, Y1, D).
gcd(X, Y, D) :-
    X > Y,
    gcd(Y, X, D).
```

למעשה זה קוד רקורסיבי, שמורץ שוב ושוב עד שמקבלים את X עם עצמו. השלישי משמאל בכלל הראשון הוא X – שמחזירים. לכן איך נריץ את זה? נריץ $gcd(7,12,S)$ על מנת לקבל לתוך המשתנה S את ה(7,12)gcd.

<u>רשימות:</u>

רשימה נכתבת בתוך סוגריים מרובעים, למשל [1,2,3]. רשימה ריקה []. ניתן להשתמש בסימון [H|T] על מנת לחלק את הרשימה לראש וזנב. דוגמה:

```
?- [a | [b,c,d]] = [a,b,c,d].
true.
```

למשל,

```
?- [H|T] = [a, b, c, d, e].
H = a,
T = [b, c, d, e].
```

H הוא תמיד האיבר הראשון וT הוא שאר הרשימה בלעדיו – היא תמיד רשימה בעצמה. ביצוע $[a|[b,c,d]]$ שקול להוספת איבר לראש הרשימה.

- **<u>אורך רשימה:</u>** נראה קוד בפרולוג שמקבל רשימה ומשתנה ומחזיר לתוך משתנה את מספר האיברים ברשימה.

```
size.pl
```
```
size([], 0).
size([H|T], N) :- size(T,N1), N is N1 + 1.
```

למעשה זה הוגדר מבסיס וצעד. הבסיס הוא רשימה ריקה, במקרה זה אורכה 0. אחרת, הצעד הוא שהאורך של $[H|T]$ הוא כמו האורך של T ועוד 1.

- **<u>קוד בפרולוג שבודק אם איבר נמצא ברשימה נתונה:</u>** החוק הראשון הוא שX נמצא ברשימה אם הוא הראש שלה. נשים לב שX מופיע פעמיים – פעם בתור מה שמחפשים ופעם בתור הראש. החוק השני הוא שהוא נמצא ברשימה אם הוא נמצא בזנב: כלומר קוראים ברקורסיה על שאר הרשימה ובודקים אם הוא שם. נשים לב כי _ פירושו "יש כאן משהו, לא אכפת לי מהו".

```
isin.pl
```
```
isin(X, [X|_]).
isin(X, [_|T]) :- isin(X,T).
```

- **<u>קוד בפרולוג שמקבל שתי רשימות ומשתנה נוסף ומחזיר במשתנה הנוסף את שרשור שתי הרשימות:</u>** במקרה הראשון שבו הרשימה הראשונה ריקה, מחזירים את הרשימה השנייה. אחרת – לוקחים את הראש של הרשימה הראשונה ושמים אותו בראש התוצאה. ומשרשרים רקורסיבית את השאר: ללא האיבר הראשון.

```
concat([],L,L).
concat([X|L1], L2, [X|L3]) :- concat(L1,L2,L3).
```

- **<u>קוד שסוכם את הרשימה:</u>** במקרה הבסיס הרשימה הריקה. אחרת, סוכמים את זנב הרשימה ומוסיפים לN את האיבר הראשון.

```
sumlis([],0).
sumlis([H|T],N) :- sumlis(T,N1), N is N1 + H.
```

- **Main:**

בפרולוג אין באמת `main` , אבל אפשר ליצור `main` ולהריץ אותו כך:

```
swipl -q -s <file name> -g <what to run>
swipl -q -s hello.pl -g main
```

hello.pl

```
main :-
    write('hello prolog').
```

תרגול 2

בתרגול הקודם למדנו לכתוב פרולוג. בתרגול הזה, נראה מה קורה מאחורי הקלעים של פרולוג. נלמד – **כיצד פרולוג מוצא תשובות**, מה קורה מאחורי הקלעים כאשר מקלידים שאילתה? מה נותר לנו לגלות – אם יש כמה כללים או כמה תשובות אפשריות, באיזה סדר פרולוג מנסה אותם? ואם אחד נכשל באמצע – מה הוא עושה?

נתבונן בדוגמה משמאל. מה התוכנית עושה? Alert (X, Level) מתאים לערך מספרי X רמת התראה: אם X<3 אזי normal, אם 6<X אזי alert2 ואחרת: alert1. ישנם שני חוקים שקובעים כיצד פרולוג מחפש:

alert.pl

```
alert(X, normal) :- X < 3.
alert(X, alert1) :- X > 3, X < 6.
alert(X, alert2) :- X> 6.
```

```
?- consult(alert).
true.

?- alert(2, Y).
Y = normal .

?- alert(2, Y).
Y = normal ;
false.
```

```
?- alert(2,Y), Y = alert1.
false.
```

1. כללים מלמעלה למטה – כדי להוכיח מטרה, פרולוג מנסה את הכללים של הפרדיקט לפי הסדר בקובץ.
2. מטרות משמאל לימין: בגוף של כלל, או בשאילתא עם פסיקים: פרולוג מוכיח את המטרות לפי הסדר.

- Backtracking: באשר מטרה נכשלת, פרולוג חוזר לנקודת ההחלטה האחרונה בה היו עוד אפשרויות ומנסה את האפשרות הבאה. אם אין אף אחת – ה"כישלון" עולה שלב אחד אחורה, עד שאין לו לאיפה לעלות.

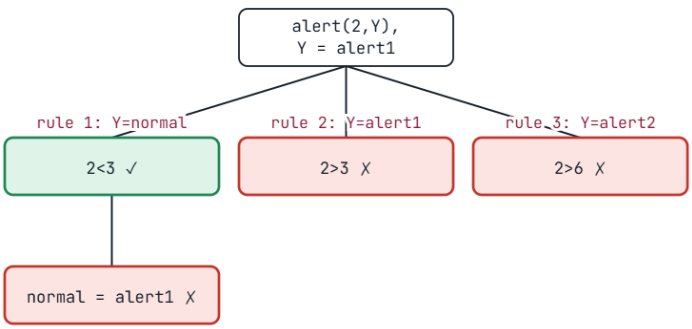
מה קורה אצלנו בדוגמה? טוענים ראשית את alert.pl. בשלב הבא, מריצים alert(2, Y): הראש X=2 מתאים עם הכלל הראשון, הגוף אכן נכון 2<3 ולכן מחזירים Y=normal. עם זאת, פרולוג זוכר שיש עוד כללים לנסות: זו נקודת בחירה. מה ההבדל בין שתי הריצות בטרמינל אם ככה? בריצה הראשונה המשתמש לחץ ENTER, מופיעה נקודה. משמעות הדבר: "מספיק, אל תחפש עוד". בריצה השנייה לעומת זאת – המשתמש לחץ נקודה פסיק ולכן ביקש אקטיבית עוד תשובה. פרולוג חוזר לנקודת ההתחלה ומנסה את הכלל 2 ואת כלל 3: שניהם נכשלים. לכן הוא מחזיר false. נשים לב שהfalse סה"כ אומר – אין תשובות נוספות.

```
?- trace.
true.

[trace]  ?- alert(2, Y), Y=alert1.
   Call: (11) alert(2, _31798) ? creep
   Call: (12) 2<3 ? creep
   Exit: (12) 2<3 ? creep
   Exit: (11) alert(2, normal) ? creep
   Call: (11) normal=alert1 ? creep
   Fail: (11) normal=alert1 ? creep
   Redo: (11) alert(2, _31798) ? creep
   Call: (12) 2>3 ? creep
   Fail: (12) 2>3 ? creep
   Redo: (11) alert(2, _31798) ? creep
   Call: (12) 2>6 ? creep
   Fail: (12) 2>6 ? creep
   Fail: (11) alert(2, _31798) ? creep
false.
```

Debug: ניתן להריץ דיבאגר של prolog ע"י הפקודה debug. אם נרצה לראות גם את מהלך הקריאות של פרולוג כאשר הוא מחפש לנו תשובה לשאילתא שלנו נוכל להשתמש בפקודה trace.

לדוגמה, הנה המעקב על הפקודה שהרצנו:



במקום לחשוב על המעקב בצורה זו, נוכל לחשוב עליו כעץ חיפוש. כך, כמו בימין. כיצד ניתן לקרוא אותו?

- השורש של העץ הוא השאילתא.
- כל ענף, הוא בחירה של כלל אחר עבור alert(2, Y). הכיתוב על הקשת אומר מה ההתאמה לראש הכלל נתנה לY.
- כל צומת, מראה מה נותר להוכיח לאחר הבחירה.

פרולוג סורק את העץ לעומק, משמאל לימין בצורת DFS: יורד בענף השמאלי ביותר עד שמצליח או נכשל, כשנכשל עולה מעלה לנק' הבחירה הקרובה ביותר וחוזר הלאה לענף הבא.

כדאי לשים לב, כי גם לאחר שהבדיקה שלו עם 2=Y נכשלה על האם 3<2, למרות זאת הוא הריץ את הבדיקה המיותרת האם 6>2. הוא לא יודע שיש קשר בין התנאים! אנחנו – עם מוח אנושי מסוגלים לעשות זאת. הוא, כמובן לא. מכאן נקבל מוטיבציה לדבר הבא.

האופרטור !: נרצה לבצע cut בעץ החיפוש על מנת שלא יתבצע backtracking מיותר. CUT מסומן ע"י האופרטור !, על מנת להשתמש בו אנחנו מבצעים "וגם" (פסיק) אחרי התנאי שאנחנו רוצים לחתוך בו את העץ. ! תמיד מחזיר לנו הצלחה. באופן פורמלי, נניח כי

$$H: -B_1, \ldots\ldots\ldots, B_n, !, B_{n+1}, \ldots\ldots, B_(n+m)$$

אזי אם $B_1, \ldots\ldots, B_n$ מתקיים אזי יתבצע החיתוך של העץ וכל הפתרונות שלא מקיימים את $B_1, \ldots\ldots, B_n$ לא יבדקו כלל בחיפוש. נרחיב מעט יותר: נניח כי פרולוג מנסה להוכיח מטרה G ובחר בשבילה בכלל H לעיל.

1. פרולוג מוכיח את $B_1, \ldots\ldots, B_n$
2. כשמגיעים ל! הוא מצליח מיד ("תמיד מחזיר הצלחה") אך ברגע זה פרולוג מתחייב לשתי בחירות:

- לכלל הנוכחי: לא ינסה את הכללים שמתחת לו עבור G. (כלומר, לא עוברים אח"כ לכללים אחרים שהוגדרו מתחתיו!)
- להצבות של $B_1, \ldots\ldots, B_n$: לא יחפש להן פתרונות אחרים.

המטרה היא: אחרי ! $B_{n+1}, \ldots\ldots, B_{n+m}$ ממשיכות כרגיל וגם backtracking בניהן מותר.

- אם משהו אחרי ה! נכשל, והbacktracking מגיע חזרה אל ה! אזי המטרה G כולה נכשלת מיד.

במילים של עץ החיפוש - ! גוזם את כל הענפים שעוד לא נבדקו, מהצומת של G ועד ל!.

נסביר בצורה יותר אינטואיטיבית: כאשר פרולוג מוכיח מטרה ויש לו כמה אפשרויות – הוא לוקח את הראשונה ומשאיר לעצמו סימנייה "לשוב לכאן אחרי זה". ה! מוחק את הסימניות שנוצרו מאז שפרולוג התחיל את הכלל הנוכחי. כלומר פרולוג לא יחזור לנסות אפשרויות אחרות שם.

<u>נתבונן בדוגמה.</u> כמה אפשרות ישנם לפרולוג להדפיס? 2*3=6 ועוד 1: כלומר סה"כ 7 אפשרויות. נשנה כעת את הקוד ונקבל

```
num(1).  num(2).  num(3).
letter(a).  letter(b).

pair(N, L) :- num(N), letter(L).
pair(0, z).
```

```
pair(N, L) :- num(N), !, letter(L).
pair(0, z).
```

במקרה זה רק 2 אפשרויות. מהן?

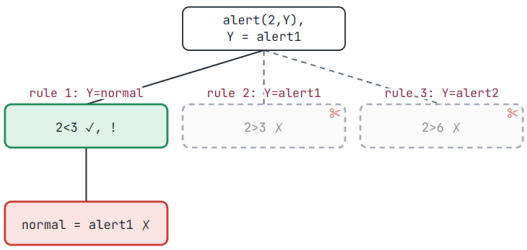
```
N=1, L=a ;
N=1, L=b ;
false.
```

נרצה להבין מה קורה מאחורי הקלעים. בדוגמה הראשונה – יש שלושה סוגי "סימניות". סימניות של הכלל – לpair יש עוד כלל. סימניות של num: אחרי N=1 יש עוד 2 ואת 3, וסימניות של letter: אחרי L=a יש L=b. **מה קורה בדוגמה השנייה?** פרולוג מתחיל את הכלל הראשון של pair ומחזיק בראשו **סימנייה 1: יש עוד כלל.** Num(N) נותן N=1 ומחזיק בראשו עוד סימנייה: **יש גם 2 ו3.** מגיעים ל!, הוא מצליח (תמיד) ומוחק את סימניות 1 ו2: כי שתיהן נוצרו מאז שהתחלנו את הכלל הנוכחי. ואז, Letter(L) נותן L=a וזו סימנייה 3: יש כאן גם עוד b שמחכה – זו סימניה שהופיעה לאחר ה! ולכן היא לא נמחקת. לכן כשלוחצים נקודה בפסיק מקבלים גם את N=1,L=b.

נעיר כי לאחר שמצאנו פתרון אחד שמקיים את התנאים שלפני ה!, הפרולוג לא מחפש עוד דרכים להגיע לאותה נקודה.

כעת נתבונן בדוגמה הקודמת – ונרחיבה: נקבל את העץ הבא.

```
alert(X, normal) :- X < 3, !.
alert(X, alert1) :- X > 3, X < 6, !.
alert(X, alert2) :- X> 6, !.
```



מה ההבדל כאן? בסוף כל כלל הוספנו !. בכלל הראשון – המשמעות היא אם X<3 אזי התשובה היא normal ואל תנסה את הכללים האחרים. מה הרווחנו כאן? יעילות! זו אותה התשובה בדיוק, עם הרבה פחות עבודה. כללים 2 ו3 יגזמו מהעץ ובכלל לא ינסו אותם.

pred.pl

```
p(1).
p(2) :- !.
p(3).
```

שאלה. נתבונן בדוגמה הבאה:

```
?- consult(pred).
true.

a) ?- p(X).
X = 1 ;
X = 2.

b) ?- p(X), p(Y).
X = Y, Y = 1 ;
X = 1,
Y = 2 ;
X = 2,
Y = 1 ;
X = Y, Y = 2.

c) ?- p(X), !, p(Y).
X = Y, Y = 1 ;
X = 1,
Y = 2.
```

ונתבונן בהרצה הבאה. נסביר אותה:

ראשית, יש שלושה דרכים להוכיח p(X): או עם העובדה p(1), או עם הכלל .!-:p(2) או עם העובדה p(3). הכלל השני אומר – p(2) נכון ואם הגעת לכאן: אל תנסה את p(3).

A: אנחנו מקבלים ראשית X=1 כי הוא מופיע ראשון, ואז עושים נקודה פסיק ומקבלים X=2 אך בגלל ה! הcut מוחק את האפשרות לנסות את p(3).

B: כאן אנחנו מריצים בשתי קריאות נפרדות את p. הנקודה שחשובה – הcut בתוך p(2) משפיע רק על הקריאה שבה הוא רץ. בתחילה, p(X) מניב X=1 ואז גם p(Y) מניב Y=1. הדרך של SWI לכתוב "שניהם 1" היא X=Y,Y=1. בהמשך, חוזרים לp(Y) – זו המטרה האחרונה עם אפשרויות. מקבלים Y=2 והCUT בתוך הקריאה הזו מונע Y=3, לכן נקבל גם X=1,Y=2. זה מעין "רקורסיה", אנחנו בתת המקרה של p(Y) כש1=X.

לאחר מכן, חוזרים לp(X) וכאן X=2 והcut מונע את X=3, ואז p(Y) מתחיל מחדש! סה"כ נקבל שוב את X=2,Y=1 וכן X=2,Y=2 . מסיימים כי לא מתקדמים לx=3.

C: כאן, ה! נמצא בשאילתא עצמה. בין זוג מטרות: מה עושים? אחרי שעוברים אותו, אסור לחפש פתרונות אחרים למה שלפניו. לכן מתחילים וp(X) מניב X=1, עוברים אותו ואז X נגעל על 1. ואז p(Y) מניב 1. מסיימים עם X=Y=1. ואז שוב, יש עוד מקום לp(Y) שמניב 2 ולכן נקבל X=1,Y=2 וזהו: מסיימים את הריצה. העקרון החשוב: **לאחר ! בשאילתא – לא משנים את הערכים שלפניו.**

דוגמה: פרדיקט שמקבל רשימה ומשתנה ומחזיר האם המשתנה נמצא ברשימה. מתבססים כאן על עקרון פשוט – או שהוא ראש הרשימה, ואז הוא ברשימה. או שניתן לפרק אותו לראש וזנב. ואז נחפש אותו בזנב (כי הוא לא הראש).

```
isin(X, [X|_]).
isin(X, [_|T]) :- isin(X,T).
```

דוגמת הפיקניק:

```
holiday(sunday, passover).

weather(sunday, good).
weather(friday, good).
weather(saturday, good).

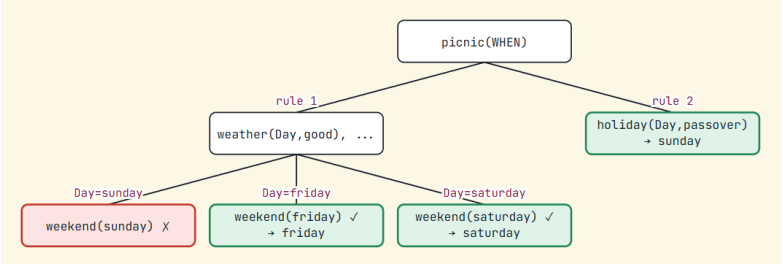
weekend(friday).
weekend(saturday).

%  We go for picnics on good weekends or in Passover
picnic(Day) :- weather(Day,good), weekend(Day).
picnic(Day) :- holiday(Day, passover).
```

נתבונן בדוגמה שמשמאל. מה התוכנית אומרת? היא מכניסה מספר עובדות. החג פסח מתקיים ביום ראשון, מזג אוויר טוב בראשון שישי ושבת. סוף השבוע הוא שישי ושבת. ואז מגדירים כלל: לפיקניק הולכים **או בסוף שבוע עם מזג אוויר טוב, או בפסח.** שני הכללים עבור פיקניק הם למעשה "או". ואז נטען את הקובץ ונבצע את השאילתה הבאה:

```
?- consult(picnic).
true.

?- picnic(WHEN).
WHEN = friday ;
WHEN = saturday ;
WHEN = sunday.
```



מתי יש פיקניק? הוא מחזיר לנו – בימי שישי, שבת או ראשון. כיצד פרולוג מוצא את התשובות? נתבונן בתרשים ונסבירו:

ראשית, לפי כלל אחד הוא מנסה את המטרה הראשונה – מזג אוויר. וישנה גם מטרה שנייה. הוא מתחיל ממטרה ראשונה – מחפש את weekend(day): עבור "יום ראשון" הוא נכשל – אין עובדה כזו. ואז הוא מוצא את שישי ושבת, ומחזיר אותם לפי הסדר. בסוף: הוא חוזר לכלל 2 – הרי כי holiday(Day, Passover) משיב Sunday ולכן מחזירים אותו.

כעת נניח ונשנה את הכללים להיות:

```
picnic(Day) :- weather(Day,good), !, weekend(Day).
picnic(Day) :- holiday(Day, passover).
```

כעת, אם נריץ אותה שאילתא כמו קודם

נקבל false. מדוע? כי ה! אומר כי עליך לקחת את היום הראשון עם מזג אוויר טוב ולהתחייב אליו. לפי רשימת הכללים – יום עם מזג אוויר טוב הוא ראשון, שלא מופיע כ"סוף שבוע": לכן אנחנו עוברים את ה! ואסור לנו לחפש ימים אחרים לweather וכן **<u>אסור לנסות את כלל 2 של פיקניק</u>** (מדוע? כי לנסות את ה"כלל הבא" היא סימניה שנקבעה לפני !). הbacktracking מגיע ל! ולכן סה"כ השאילתא נכשלת ומוחזר false. נשים לב כי קיבלנו שקר למרות שיש ימים שמתאימים לפיקניק – זו דוגמה לא טובה. המיקום של ! הוא קריטי. הוא התחייב כאן לבחירה של weather לפני שבדק אם היא מתאימה כלל.

לעומת זאת אם היינו משנים את הכללים להיות:

```
picnic(Day) :- weather(Day,good), weekend(Day), !.
picnic(Day) :- holiday(Day, passover).
```

כאן יוחזר לנו WHEN=Friday. כאן אנחנו מחפשים יום עם מזג אוויר טוב שהוא גם סוף שבוע. ברגע שמצאת אחד, עצור. מה קורה? מזג אוויר נותן Sunday: אבל weekend(Sunday) נכשל. עוד לא הגענו ל! ולכן מותר באק-טראקינג רגיל. Weather יניב Friday: גם weekend(Friday) יעבור ולכן נסיים. אין עוד אפשרויות כי שבת נגנז כי זו סימניה שהוגדרה לפני ה!, ונמחקת. **ההבדל הוא שבדוגמה הקודמת ה! היה לפני הבדיקה שאכן היום הוא סוף שבוע. לכן התחייבנו ליום שלא נבדק עד הסוף. כאן הוא לאחר הבדיקות, לכן אם אכן יש יום שעונה על הדרוש – הוא יבחר.**

ומה אם הכללים יראו כך?

```
picnic(Day) :- !, weather(Day,good), weekend(Day).
picnic(Day) :- holiday(Day, passover).
```

כאן אנחנו שמים את ! בהתחלה. המשמעות היא: ! הוא המטרה הראשונה, אין לפניו כלום. לכן הוא מתחייב לדבר אחד בלבד: לכלל הזה. מה הכוונה? "אל תנסה את כלל 2 – לעולם". לכן **! בתחילת הכלל משמעותו: התחייב לכלל זה.** לכן אנחנו נשאלים למעשה על ימים שהם סוף שבוע עם מזג אוויר טוב בלבד – פסח לא ייבדק. לכן יוחזר לנו שישי ושבת. נשים לב שזה הגיוני – אנחנו מבצעים את הכללים עד לסיום הכלל הראשון כי "הסימניות שלהם" הופיעו לאחר ה!. לעומת זאת – הסימנייה היחידה שכן הופיעה לפני ה!: היא הסימנייה של הכלל השני.

<u>שלילה</u>

נניח ונרצה לכתוב פרדיקט כמו: "משה אוהב את כל החיות, חוץ מנחשים". זה למעשה שלילה – "X הוא חיה, וגם X אינו נחש". נוכל לכתוב את הכלל $likes(moshe, X) :- animal(X)$. אבל הבעיה היא – שפרדיקט זה כולל גם נחשים. נרצה לבטאו בצורה אחרת. למשל:

- אם X הוא נחש, אז "משה אוהב את X" – לא נכון.
- אם X הוא חיה, משה אוהב אותו.

בפרולוג על מנת לומר שמשהו איננו נכון משתמשים בfail. זו מטרה שתמיד נכשלת. לעומת זאת – true היא מטרה שתמיד מצליחה.

נראה את זוג הכללים:

```
likes(moshe, X) :- snake(X), ! , fail.
likes(moshe, X) :- animal(X).
```

הצירוף $!, fail$: כאשר יש תנאי משמאל – פירושו: "אם התנאי מתקיים, אזי הפרדיקט נכשל. ואל תנסה דרכים אחרות". זו הדרך לבטא לא בפרולוג. נניח כעת כי העובדות הן:

$$snake(cobra), animal(cobra), animal(dog)$$

נריץ $likes(moshe, cobra)$ על הכלל. ראשית, עוברים את כלל 1: *snake(cobra)*. משם – עוברים את ! ולכן כלל 2 לא ינוסה בכלל. ואז נכשל. הבאק-טראקינג מגיע ל! ולכן סה"כ $likes(moshe, X)$ נכשל כולו. לעומת זאת, אם נריץ $likes(moshe, dog)$ אזי הכלל הראשון *fail* נכשל – זה לפני שהגענו ל! ולכן עוברים לבאק-טראקינג רגיל לכלל 2. בכלל 2 מצליחים ולכן מחזירים *true*.

המשמעות: דאגנו כי אם החיה היא אכן נחש – אתה בוודאות לא יכול ללכת לכלל אחר, ואתה בוודאות תכשל. אם החיה לא נחש – היא לא תעבור את הבדיקה הראשונית לפני ! ותוכל להגיע לבדיקה השניה: שם היא תצליח.

דוגמה. נגדיר פרדיקט אשר נכשל אם קיבל שני אלמנטים זהים – ואחרת הוא מצליח.

```
different(X, X) :- !, fail.
different(X, Y).
```

בכלל הראשון: אם האותיות X, X מתאימות זו לזו? התשובה היא כן. במקרה זה: עוצרים, לא עוברים לכלל אחר ומחזירים שקר. אחרת – אם אנחנו עם האותיות X, Y אזי הן שונות מראש ולכן נאשר.

- האופרטור not: אופרטור מובנה בתוך פרולוג שמוגדר באופן הבא:

$$not(P) := P, !, fail; true$$

מה המשמעות שלו? הוא אמת אם ורק אם P הוא שקר. לכן למשל את הפרדיקט כי משה אוהב את כל החיות פרט לנחש ניתן להגדיר:

```
likes(moshe, X) :- animal(X), not snake(X).
```

הבהרה. הסינטקס של ; הוא "או" בתוך גוף של כלל. למשל את פרדיקט השלילה ניתן לכתוב גם כך:

```
not(P) :- (P, !, fail)  ;  (true).
%         צד ימין        צד שמאל
```

ישנן 2 אפשרויות ופרולוג מנסה קודם את הצד השמאלי. כאשר הוא נכנס אליו הוא מכניס סימנייה: אם נצטרך, יש גם את הצד הימני. למה זה עובד? אם P אכן נכון – אזי הוא יקבל את הערך אמת, נתקדם ל! ואז נכשל ונחזיר שקר. בדיוק ההפך מP. אחרת, P הוא לא נכון: ואז לא נכנס למחוק אותו בכלל – ישר נחזיר $true$.

<u>הנחת העולם הסגור</u>

הנחת העולם הסגור: התוכנית מכילה את כל האמת. כל דבר שאי אפשר להוכיח מהתוכנית, נחשב כשקר.

זו ההנחה גם מאחורי $false$ של פרולוג. כאשר פרולוג עונה "שקר" המשמעות המדויקת הינה – "לא הצלחתי להוכיח את זה מהעובדות והכללים שיש לי". הוא לא בדק את העולם האמיתי – את התוכנית בלבד. מכאן גם, נוצרת בעיה בפרדיקט של השלילה: על כל דבר שלא בשפה – הוא יחזיר אמת. אין זה שקול לכך שהטענה נכונה. לכן, הפרדיקט not **לא** שקול לשלילה לוגית מתמטית.

<u>נתבונן בדוגמה.</u>

res.pl

```
good_standard(hamburger).
good_standard(pizza).

expensive(hamburger).

recommended(X) :- not(expensive(X)).
```

```
?- consult(res).
true.

?- good_standard(X), recommended(X).
X = pizza.

?- recommended(X), good_standard(X).
false.
```

אנחנו מגדירים המבורגר ופיצה, וטוענים כי המבורגר הוא יקר. אנחנו ממליצים על דברים שאינם יקרים. אנחנו מבצעים זוג שאילתות. שתיהן שואלות – "מה ברמה טובה וגם מומלץ?". הרי, מבחינה לוגית מתקיים $A \wedge B = B \wedge A$ ולכן אין משמעות למי מופיעה קודם בשאילתא. אבל, בשאילתא הראשונה אכן מוחזר לנו פיצה כמו שציפינו אך בשנייה לא. מדוע? כי בסדר השני $not(expensive(X))$ נקרא כאשר X הוא עוד פתוח. אז זה שואל: "האם יש משהו יקר?" – התשובה היא שיש – המבורגר. לכן, יוצא "שום דבר לא מומלץ".

מכאן נפתח כלל אצבע – קוראים לNOT רק כאשר כל המשתנים בתוכו כבר קיבלו ערכים. לכן נסדר את המטרות כך שהמטרות החיוביות שנותנות ערכים יופיעו לפני השלילה.

תרגול 4

ראינו בתחילת הקורס כי שפות תכנות מתחלקות ל-2 סוגים. סוג ראשון הוא שפות הצהרתיות – המיקוד של השפה הוא במה המחשב צריך לעשות, וישנן שפות מצוות שם המיקוד הוא באיך המחשב צריך לעשות. הסוג הראשון, מתחלק לשני סוגים: שפות פונקציונליות – בהן פונקציות קוראות לפונקציות. לדוגמה: Ocmal וישנן שפות לוגיות עם חוקים לוגיים כמו פרולוג.

SML היא שפת תכנות פונקציונלית, עם בדיקת טיפוסים בזמן קומפילציה והסקת טיפוסים. אלו ראשי תיבות של Standard Meta language.

- קבצי SML נגמרים בסיומת sml. למשל hello.sml.

קוד של SML כותבים בקבצים רגילים, נתבונן בהרצה הבאה:

```
$ cat hello.sml
print "Hello, world!\n";
$ sml
Standard ML of New Jersey [Version 110.99.8; 64-bit; April 25, 2025]
- use "hello.sml";
[opening hello.sml]
Hello, world!
val it = () : unit
```

תחילה, אנחנו מבצעים cat ומדפיסים את תוכן הקובץ. Print היא הפונקציה להדפסת מחרוזות – היא מקבלת מחרוזת ומדפיסה אותה למסך. **נשים לב שאין סוגריים.** בSML פשוט מפעילים פונקציה על ארגומנט פשוט ע"י כתיבתם זה אחרי זה.

הפקודה sml בטרמינל מבצעת כניסה לסביבת עבודה. היא מדפיסה את הגרסה ואז מחכה לקלט עם הסימן – בתחילת השורה.

סביבת עבודה – הכוונה לREPL: לולאה שקוראת ביטוי שכתבתם, מחשבת אותו ומדפיסה אותו וחוזרת לחכות.

הפקודה "use "hello.sml - היא הפקודה להרצת קובץ מתוך סביבת העבודה. היא קוראת את הקובץ ומריצה את כל השורות שבו, כאילו הקלדנו אותו לבד.

- בשורה הראשונה, [opening hello.sml] SML מודיעה שהיא פותחת את הקובץ.
- בשורה השנייה, זו ההדפסה עצמה. מה-PRINT שבקובץ
- unit: () Val it= זהו הערך שUSE מחזירה, תמיד.

בSML כל ביטוי מחזיר ערך – גם פקודה ש"רק עושה משהו". כשאין ערך מעניין להחזיר, מחזירים את () – הטיפוס שלו נקרא unit ויש לו ערך אחד בלבד. זה כמו VOID בC.

```
$ cat hello.sml
print "Hello, world!\n";
$ mlton hello.sml
$ ./hello
Hello, world!
```

את אותו הקובץ ניתן גם לקמפל לקובץ הרצה. MLTON מקמפלת את הקובץ – אין פלט ונוצר קובץ הרצה חדשה בשם hello.sml. מריצים אותו – והוא מדפיס מה שציפינו שידפיס.

ההבדל בין שתי הדרכים:

	SML/NJ (sml)	MLton
איך עובדים	סביבה אינטראקטיבית, מקלידים ורואים תשובה	מקמפלים קובץ שלם, ואז מריצים
מראה טיפוסים וערכים	כן (val it = …)	לא, רק מה שמדפיסים
מתאים ל	ניסויים ולמידה	תוכנית סופית ומהירה

ביטויים

```
- 3 + 4;
val it = 7 : int
```

נפתח בדוגמה, משמאל. הקלדנו ביטוי תחת – (כלומר זה אנחנו מקלידים בSML) והשורה נגמרת תמיד ב;. הנקודה פסיק אומרת לסביבה – סיימתי, תתחילי לחשב. בלעדיה: SML חושבת שהביטוי ממשיך בשורה הבאה ומחכה. לתשובה של SML יש שלושה חלקים:

חלק	בדוגמה	משמעות
המשתנה	`it`	השם שבו נשמרה התוצאה
הערך	`7`	התוצאה של החישוב
הטיפוס	`int`	מספר שלם

כלומר מגדירים משתנה it, שהוא יקבל את התוצאה 7 – ולאחר הנקודתיים מגדירים את הטיפוס. הוא מסוג טיפוס int.

מה זה it? כשמקלידים ביטוי מבלי לתת לו שם – SML שומרת את התוצאה במשתנה מיוחד בשם it. לאחר מכן ניתן להשתמש בו כשאנחנו כותבים לSML. למשל כך:

```
- 3 + 4;
val it = 7 : int
- it * 2;
val it = 14 : int
```

התשובה תמיד תהיה $name = value : type$.

```
- val x = 7;
val x = 7 : int
```

משתנים: נתבונן בדוגמה משמאל. כאן, במקום ביטוי בלי שם כתבנו מפורשות val x=7. התשובה, תהיה באותו מבנה רק שהפעם היא לא תשתמש במשתנה it, אלא במשתנה שהגדרנו. **כאן יש הבדל חשוב Cמ ומפייתון:** X הוא שם לערך, לא "קופסה" שניתן לשנות את תוכנה. לכן אין בSML פקודה כמו X=X+1. למשל, אם נכתוב כעת val x=10 זה לא ישנה את הx הישן אלא יצור חדש בשם x שיסתיר את הקודם. קוד שכבר השתמש בX הישן ימשיך לראות 7. לכן אם נסכם: שורת קוד זו נותנת את השם x לערך 7 – את הערך אי אפשר לשנות.

```
- val x = 7;
val x = 7 : int
- val y = 2 + 3;
val y = 5 : int
- val z = x + y;
val z = 12 : int
```

כמו כן, ניתן להמשיך ולהרחיב עם הדוגמה הבאה –

מה קורה כאן? תוכנית רגילה. נשים לב כי בשורה 3 קודם מחשבים את 2+3 ורק אז את התוצאה 5 שומרים במשתנה Y. SML לא זוכרת את 2+3.

בכל שורה SML מדפיסה את הערך שחושב ואת הטיפוס – במקרה שלנו הכל INT.

Real

```
- val x = 7.0;
val x = 7.0 : real
- val y = x + 2.0;
val y = 9.0 : real
```

מספר עם נקודה עשרונית הוא מטיפוס real: ממשי.

- פעולות על שני real הן מטיפוס real (כלומר תוצאת הפעולה).

ישנם כמה אופרטורים של חילוק:

אופרטור	על מה	דוגמה
`/`	רק `real`	`9.0 / 7.0` = `1.28571428571`
`div`	רק `int` (חילוק שלם)	`9 div 7` = `1`
`mod`	רק `int` (שארית)	`9 mod 7` = `2`

נשאל את עצמנו – האם SML ממירה טיפוסים לבד? לא!
בדוגמה משמאל – X הוא ממשי אך 2 הוא שלם. ניסינו לחבר ממשי ושלם: זו שגיאה. בשפת C זה היה עובד כי 2 היה מומר למספר ממשי, באופן אוטומטי. כאן SML לא ממירה טיפוסים לבד! זו נחשבת **שגיאה בזמן קומפילציה** – הקוד לא רץ בכלל. הפתרון הוא לכתוב במקום 2.0+x.

```
- val x = 7.0;
val x = 7.0 : real
- val z = x + 2;
stdIn:1.6-1.14 Error: operator and operand do not agree [overload - b
  operator domain: real * real
  operand:         real * 'Z[INT]
  in expression:
    x + 2
```

טיפוסי string וchar:

ב-SML יש טיפוס מסוג string – מחרוזת כותבים תחת זוג מרכאות כפולות "..".

```
- "Hello SML";
val it = "Hello SML" : string
```

כמו כן, ישנו טיפוס char. תו בודד. כותבים אותו תמיד עם # לפני זוג המרכאות.

```
- val myChar = #"H";
val myChar = #"H" : char
```

נשווה:

כתיב	טיפוס	מה זה
"H"	string	מחרוזת באורך 1
#"H"	char	תו בודד

אלו – לא אותו הדבר! אם נשתמש אחד במקום השני: שגיאה.

שרשור מחרוזות: האופרטור ^ משרשר בין המחרוזת שמופיעה לשמאלו, למחרוזת שמופיעה מימינו. לדוגמה:

```
- val part1 = "Hello, ";
val part1 = "Hello, " : string
- val part2 = "World!";
val part2 = "World!" : string
- val fullGreeting = part1 ^ part2;
val fullGreeting = "Hello, World!" : string
```

- הערה: ב-SML + לא עובד בין זוג מחרוזות כמו שעובד בפייתון או בג'אווה. לכל פעולה יש אופרטור משלה. + למספרים, ^ למחרוזות.

פונקציית string.sub: הפונקציה $String.sub(s,i)$ מחזירה את התו במיקום i של המחרוזת S. הספירה מתחילה מ0 כמובן.

```
- val word = "SML";
val word = "SML" : string
- String.sub (word, 0);
val it = #"S" : char
```

פונקציית size: מחזירה את האורך של מחרוזת. לדוגמה

```
- size "Hello";
val it = 5 : int
```

הפונקציה String.extract(s, start, len): מחזירה חלק מהמחרוזת, החל ממיקום start – הארגומנט השלישי קובע את ה"חלק".

- SOME 2 - קח בדיוק 2.
- NONE: קח עד הסוף.

```
- val text2 = "Standard ML 123";
val text2 = "Standard ML 123" : string
- String.extract (text2, 9, SOME 2);
val it = "ML" : string
- String.extract (text2, 9, NONE);
val it = "ML 123" : string
```

פונקציות

ב-SML יש טיפוס מסוג פונקציה. למשל: fun היא מילת המפתח להגדרת פונקציה, תמיד. Square היא שם הפונקציה. X הוא הפרמטר של הפונקציה, נשים לב שאין סוגריים ואין טיפוס. לאחר ה=: מה הפונקציה מחזירה. אין שום return, הגוף הוא ביטוי והערך שלו הוא ערך ההחזרה.

```
- fun square x = x * x;
val square = fn : int → int
```

נתבונן בתשובה שהוחזרה: גם פונקציה היא ערך! Val בהתחלה – ואז שמה, והטיפוס הוא פונקציה שהולכת משלם לשלם ולכן $fun: int \to int$ ולכן זהו גם הטיפוס. הערך במקרה זה הוא fn.

המסקנה שלנו – בSML יש טיפוס ממש מסוג פונקציה. זה חדש ולא הכרנו זאת עד היום – פונקציה היא ערך. כמו הערך "7" למשל. ויש לה גם טיפוס. במקום להדפיס את הקוד שלה, SML מדפיסה רק fn.

נשאלת השאלה, ובצדק: כיצד SML ידעה כי הפונקציה מחזירה שלם? זוהי הסקת טיפוסים. * הוא אופרטור מועמס – אותו סימון מוגדר גם לint,real ולכן בכל שימוש ב* צריכה SML להחליט באיזה טיפוס מדובר. בפונקציה שלנו, של העלאה בריבוע: אין ל sml דרך לדעת מה הייתה הכוונה: אפשר להכפיל ולהעלות בריבוע גם מס' ממשי – לכן ישנו כלל ברירת מחדל: **אופרטור מועמס שלא נקבע לו טיפוס מקבל באופן דיפולטיבי את int.**

כיצד מפעילים פונקציה? נתבונן בדוגמאות משמאל להמחשה. נשים לב כי הסוגריים במקרה השני הכרחיים. החישוב מחשב קודם את מה שבתוך הסוגריים – ורק אז מקבל ערך, ויוצא לבצע את העלאה בריבוע החיצונית.

```
- square 3;
val it = 9 : int
- square (square 3);
val it = 81 : int
```

בעקרון – פונקציות הן lambda expressions.

```
- fn x ⇒ x * x;
val it = fn : int → int
```

פונקציות בלי שם: למשל -

זו פונקציה בתחשיב למדה. פונקציה ללא שם. קוראים אותה "הפונקציה שמקבלת X ומחזירה X*X". החץ מפריד בין הפרמטר לגוף. כמו כל ביטוי ללא שם, היא נשמרת בit.

כמו כן, כמובן שניתן לתת לה שם עם val:

```
- val square = fn x ⇒ x * x;
val square = fn : int → int
```

להלן בעצם שתי דרכים לכתוב אותו דבר:

```
fun square x = x * x;          (* הקיצור *)
val square = fn x ⇒ x * x;     (* מה שהוא אומר בפועל *)
```

פונקציית כפל: פונקציה שמקבלת שני פרמטרים. נתבונן משמאל:

```
- fun mul x y = x * y;
val mul = fn : int → int → int
- mul 3 4;
val it = 12 : int
```

נשים לב לטיפוס. זהו החלק החשוב. נשאלת השאלה – למה לא כתוב משהו בסגנון של "מקבלת INT 2"? החלק החשוב הוא שהחץ מתקבץ מימין. כלומר זה שקול ל

$$int \to (int \to int)$$

מה זה אומר בפועל? Mul היא פונקציה שמקבלת INT אחד – ומחזירה פונקציה מINT לINT. הקריאה mul 3 4 היא למעשה 4 (mul 3).

מדוע זה כך? לפי עקרון בSML – לפיו כל פונקציה מקבלת ארגומנט אחד בדיוק. כלומר: הפונקציה הראשונה מחזירה 3y, והשנייה מקבלת y ומציבה בו את 4 ולכן נקבל 12.

<u>If then else:</u> קצת תנאים. Foo היא פשוט פונקציה שמחזירה את הגדול מבין השניים. בSML, IF הוא ביטוי שמחזיר ערך ולא פקודה. לכן: ELSE הוא חובה – הביטוי הזה חייב להחזיר משהו בכל מקרה. שני הענפים חייבים להיות מאותו טיפוס.

```
- fun foo x y = if x > y then x else y;
val foo = fn : int → int → int
- foo 5 3;
val it = 5 : int
```

פונקציה המקבלת פונקציה:

```
fun sum_if_true test first second =
  (if test first then first else 0) +
  (if test second then second else 0);
```

```
val sum_if_true = fn : (int → bool) → int → int → int
```

נתבונן בדוגמה. הפונקציה הראשית שלנו מקבלת שלושה דברים: היא מקבלת פונקציית בדיקה test, ושני מספרים. היא מחזירה את סכום המספרים שעוברים את הבדיקה. מס' שלא עובר נחשב כאפס.

נפרק מדוע זה הטיפוס, באמצעות הטבלה הבאה:

פרמטר	טיפוס	למה SML הסיקה את זה
`test`	`int → bool`	מפעילים אותה (`test first`), והתוצאה משמשת כתנאי ב-`if`, ולכן `bool`
`first`	`int`	בענף ה-else מחזירים `0`, ושני הענפים מאותו טיפוס
`second`	`int`	אותה סיבה
החזרה	`int`	חיבור `+` של שני int

רקורסיה:

```
- fun f n = if n = 1 then 1 else n * f(n-1);
val f = fn : int → int
- f 4;
val it = 24 : int
```

במקום לולאות. בSML אין לולאות for or while במובן הרגיל, כי אין משתנה שניתן לקדם אותו. במקום זה משתמשים ברקורסיה. נתבונן בפונקציה הבאה שמחשבת עצרת:

פונקציות גנריות:

```
- fun first_if_true test x y = if test x then x else y;
val first_if_true = fn : ('a → bool) → 'a → 'a → 'a
```

פונקציות שעובדות על כל טיפוס שהוא. למשל: הפונקציה הבאה מחזירה X אם עבר את הבדיקה ואחרת את Y.

מה זה 'a? נשים לב שבקוד אין שום פעולה שיכולה לגלות את הערך של X או Y. לא מחברים אותו, לא כופלים אותו ולא משווים לאפס. רק מעבירים אותו לפונקציה TEST. לכן SML מסיקה כי X יכול להיות מכל טיפוס שהוא. היא מסמנת זאת במשתנה טיפוס 'a. קוראים לו "אלפא". המשמעות: זהו טיפוס כלשהו - אבל עם אותו טיפוס בכל המקומות בהם כתוב 'a.

```
- fun f x y = x + y;
val f = fn : int → int → int
- f 5;
val it = fn : int → int
```

מה יקרה אם נתן לפונקציה רק חלק מהארגומנטים שהיא זקוקה להם?

נתנו הרי לf במקרה הזה רק ארגומנט אחד – את 5. מדוע לא הוחזרה לנו שגיאה? נשים לב שהקריאה החזירה פונקציה שמקבלת מס' שלם ומחזירה מס' שלם. הקריאה יצרה פונקציה! הרי נזכר כי ככה זה עובד בפועל – ומה קיבלנו? הארגומנט הראשון נהפך לקבוע ובידינו הפונקציה y+5. ניתן גם לשמור את הפונקציה בשם.

זה נקרא **הפעלה חלקית** - שימושי כאשר רוצים להכין פונקציה עם חלק מהפרמטרים כקבועים. למשל f 1 היא הפונקציה של הוספת 1.

שגיאות ואזהרות

יש שני סוגים של בעיות:

	Error	Exception
מתי	לפני ההרצה	בזמן ההרצה
סיבה טיפוסית	טיפוסים לא מתאימים	ערך בעייתי (חילוק ב-0, רשימה ריקה...)
בדוגמה	`mul 3 4`	`is_a_multiple 8 0`

1. ERROR: שגיאת קומפילציה. מתגלה לפני ריצת הקוד. למשל – הגדרנו פונקציה שמקבלת שני REALS ובפועל הבאנו שני INT. SML גילתה זאת לפני שחישבה משהו: היא קיבלה אי התאמה בטיפוס.
2. Exception: שגיאת זמן ריצה. מתגלה תוך כדי ריצת הקוד. למשל – חלוקה באפס. כאן אין בעיות טיפוסים, לכן זה מתגלה רק בזמן הריצה.

Options: דרך יעילה ומעניינת בSML לטפל בשגיאות זמן ריצה שנוצרות מקלט לא נכון לפונקציה הוא באמצעות שימוש בOPTIONS. OPTION הוא מבנה נתונים שיש בו שתי אפשרויות: NONE או SOME. ראינו זאת קודם –

1. NONE: "אין ערך"
2. SOME V: יש ערך, והוא v.

כלומר הרעיון הוא – נחזיר ערך שעטוף בSOME אם קיים, ואם לא קיים נחזיר שגיאה.

ולהלן ההבדל – לפני שמחלקים בודקים אם Y=0 ואם כן מחזירים NONE. אחרת – מחזירים את התוצאה, עטופה.

לפני חריגה:

```
- fun is_a_multiple x y = (x div y) = 0;
val is_a_multiple = fn : int -> int -> bool
- is_a_multiple 8 0;
uncaught exception Div [divide by zero]
  raised at: stdIn:1.20-1.32
```

אחרי option:

```
- fun divide x y =
    if y = 0 then NONE else SOME (x div y);
val divide = fn : int -> int -> int option
- divide 8 0;
val it = NONE : int option
```

הטיפוס int option: הפונקציה לא מחזירה ערך שלם INT. אלא INT OPTION. זה "אולי INT". הטיפוס עצמו מודיע למי שקורא לפונקציה שיתכן שאין תוצאה . מי שמשתמש בתוצאה, חייב לטפל בשני המקרים: ובדיקת הטיפוסים לא תתן לו לשכוח. כלומר עליו לבדוק גם מצב לNONE וגם מצב לSOME.

מבני נתונים

Tuple: קבוצה סדורה של כמה ערכים, בסוגריים מופרדים ובפסיקים. הערכים יכולים להיות מטיפוסים שונים. הטיפוס נכתב עם * בין הטיפוסים באשר משמאל לו זה הטיפוס של הראשון ומימין לו הטיפוס של השני. הסדר חשוב!

```
- val a_tuple = (3, "three");
val a_tuple = (3,"three") : int * string
- val another_tuple = (3, "four", 5.0);
val another_tuple = (3,"four",5.0) : int * string * real
```

ניתן גם לפרק TUPLE למרכיביו כך:

```
- val (x, y) = a_tuple;
val x = 3 : int
val y = "three" : string
```

- הפונקציה complexsum מחברת שני מספרים מרוכבים, כאשר כל אחד מהם מיוצג כזוג – אחד ממשי ואחד חלק מדומה. היא מקבלת $(a_1, a_2), (b_1, b_2)$ ומחזירה $(a_1 + b_1, a_2 + b_2)$

זהו כמובן הטיפוס – מקבלת 2 זוגות. ומחזירה זוג:

```
int * int → int * int → int * int
```

רשימות: מדובר ברצף של ערכים **מאותו** טיפוס. רשימה נכתבת ע"י סוגריים מרובעים. הטיפוס הוא טיפוס האיברים ואחריו המילה list. לדוגמה:

```
- val arr = [5, 10, 20, 100];
val arr = [5,10,20,100] : int list
- val languages = ["OCaml", "Java", "C"];
val languages = ["OCaml","Java","C"] : string list
```

- בSML רשימות הן immutable - כלומר לא ניתנות לשינוי. אי אפשר לשנותן אחרי שנוצרו. כל פעולה על רשימה (הוספה, סינון, היפוך וכו') מחזירה רשימה חדשה: והמקורית נשארת כפי שהיא.

מספר פעולות על רשימות שכדאי להכיר:

פעולה	מה עושה	בדוגמה
`List.length`	אורך הרשימה	4
`List.nth (l, i)`	האיבר במקום `i` ברשימה (לפי אינדקס, מ-0)	מקום 0 = 5
`List.rev`	Reverse: רשימה חדשה בסדר הפוך	`[100,20,10,5]`
`List.find`	Find: האיבר **הראשון** שמקיים את התנאי	`SOME 20`
`List.filter`	**כל** האיברים שמקיימים את התנאי	`[20,100]`

- List. Find מחזירה את האיבר הראשון שמקיימים את התנאי לעומת filter שמחזירה את **כל** מי שמקיים את התנאי. חשוב לשים לב.
- Find מחזירה SOME 20. מדוע? כי יתכן שאף אחד לא מקיים את התנאי – ואז מחזירים NONE. לכן משתמשים פה בOPTION.

חיבור רשימות:

אופרטור	צד שמאל	צד ימין	מה עושה
`@`	רשימה	רשימה	מחבר שתי רשימות (append)
`::`	**איבר**	רשימה	מוסיף איבר אחד לראש הרשימה (cons)

נראה לזה דוגמה –

```
- val arr = [5, 10, 20];
val arr = [5,10,20] : int list
- arr @ [100, 200];
val it = [5,10,20,100,200] : int list
- 0 :: arr;
val it = [0,5,10,20] : int list
- 1 :: 2 :: 3 :: [];
val it = [1,2,3] : int list
```

- פונקציית map: List.map מקבלת כקלט זוג ארגומנטים – פונקציה, רשימה. היא מפעילה את הפונקציה על כל איבר ברשימה ומחזירה רשימה חדשה עם האיברים לאחר השינוי. כמובן – לא משנה את הרשימה המקורית. לדוגמה:

```
- List.map (fn x => x*x) [1, 2, 3];
val it = [1,4,9] : int list
```

Pattern Matching

הרעיון – במקום לכתוב if שבודק כיצד נראה הקלט, כותבים כמה הגדרות לפונקציה: אחת לכל צורה אפשרית של הקלט. ההגדרות מופרדות ע"י |, כמו בדוגמה משמאל. כאשר קוראים לפונקציה SML בודקת את התבניות לפי הסדר מלמעלה למטה ומשתמשת בראשונה שמתאימה.

```
- fun get_first_array [] = 0
    | get_first_array (first :: rest) = first;
val get_first_array = fn : int list -> int
- get_first_array [5,20,100];
val it = 5 : int
```

כך למשל נוכל לכתוב עצרת:

```
- fun factorial 0 = 1
    | factorial n = n * (factorial (n-1));
```

Record Type

- ניתן להגדיר TYPE חדש בSML אשר מכיל מס' שדות מטיפוסים שונים. טיפוס כזה נקרא record.

```
type <record-name> =
  { <field> : <type>,
    <field> : <type>,
    ... }
```

TYPE היא מילת המפתח – ו<record-name> הוא שם הטיפוס החדש. וכן: ניתן בסוגריים המסולסלים להגדיר את רשימת השדות. לכל שדה ישנו טיפוס ושם.

	tuple	record
דוגמה	`(3.0, ~4.0)`	`{x = 3.0, y = ~4.0}`
טיפוס	`real * real`	`{x : real, y : real}`
גישה	לפי מקום	לפי שם השדה

מה ההבדל אם כך מTUPLE? הוא דומה – כי מאגד כמה ערכים מטיפוסים שונים. אבל בTUPLE ניגשים לפי מיקום וכאן ניגשים לפי שם. זה יותר דומה לstruct בC.

- הערה: את המספר 4- כותבים בSML ע"י 4~.

כיצד ניגשים לשדה X של P? ע"י #. כך - p x#.

Variant type: נניח שנרצה ליצור אובייקט שיהיה מכמה סוגים שונים של צורה. יש לנו עיגול, מלבן, וכל אחד מהם מטיפוס אחר. אבל נרצה למשל רשימה של צורות וברשימה כל האיברים צריכים להיות מאותו טיפוס. לשם כך נצטרך ליצור טיפוס חדש variant שיכול להיות מכמה טיפוסים שונים.

```
type <variant> =
  | <Tag> [ of <type> [* <type>] ... ]
  | <Tag> [ of <type> [* <type>] ... ]
  | ...
```

כיצד קוראים את זה? כל שורה שמתחילה בן ("או") היא אפשרות אחת. <Tag> הוא שם האפשרות. Of <Type> זה איזה מידע האפשרות הזו נושאת – הסוגריים המרובעים בתבנית אומרים שזה אופציונלי. ה<type>* אומר שניתן לשאת כמה ערכים.

ככה נממש זאת:

```
datatype scene_element =
    Circle of circle_desc
  | Rect of rect_desc
  | Segment of segment_desc;
```

דוגמה לקוד שמוחק כל זוג ערכים רצופים שהם זהים: כלומר – אם יש שני איברים זהים אחד אחרי השני, מוחקים אחד מהם:

```
fun compressString [] = []
  | compressString [a] = [a]
  | compressString (a :: (b :: rest)) =
      if String.compare (a, b) = EQUAL then compressString (b :: rest)
      else a :: compressString (b :: rest);
```

<u>עצים</u>

עץ בינארי נגדיר באופן הבא: זהו variant רקורסיבי. יש שתי אפשרויות: אחת, היא שהעץ ריק. אחרת – מדובר בקודקוד שנושא שלישייה:

```
datatype 'a binary_tree =
    Empty
  | Node of 'a * 'a binary_tree * 'a binary_tree;
```

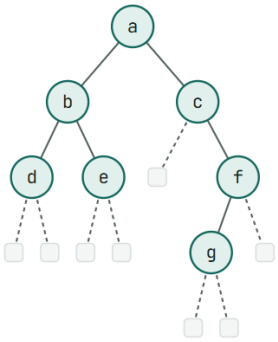
1. ערך מטיפוס 'a : תוכן הקודקוד.
2. תת עץ שמאלי
3. תת עץ ימני

נשים לב כי ההגדרה רקורסיבית – binary_tree מופיע בהגדרה של עצמו. וכן כל הערכים בעץ מאותו טיפוס – גנרי – 'a.

למשל. כך יוצרים עץ בינארי הבא:

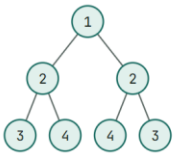
```
val example_tree =  Node(#"a", Node(#"b", Node(#"d", Empty, Empty), Node(#"e", Empty, Empty)),
Node(#"c", Empty, Node(#"f", Node(#"g", Empty, Empty), Empty)));
```

ונקבל את העץ לעיל:

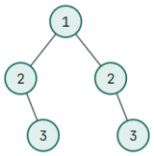


תרגיל. כתוב תוכנית המקבלת שורש של עץ בינארי, ומחזירה האם הוא סימטרי.

הגדרה: נאמר כי עץ בינארי T הוא סימטרי אם הוא תמונת מראה של עצמו – כלומר אם מעבירים קו ישר דרך השורש, מקבלים תמונת מראה של מבנה העץ בין הצד הימני לשמאלי. דוגמאות:



עץ סימטרי



עץ לא סימטרי

<u>פתרון:</u> זו הפונקציה, ונתעכב על להסביר אותה.

```
fun is_symmetric tree =
  let
    (* Check if two trees are mirror images of each other *)
    fun is_mirror (Empty, Empty) = true
      | is_mirror (Node(_, l1, r1), Node(_, l2, r2)) =
          is_mirror (l1, r2) andalso is_mirror (r1, l2)
      | is_mirror _ = false
  in
    case tree of
      Empty ⇒ true
    | Node(_, l, r) ⇒ is_mirror (l, r)
  end;
```

הפתרון בנוי משתי פונקציות. הפונקציה הראשית – ופונקציה משנית בשם is_mirror. ראשית, מה זה המבנה של let.. in.. end: זהו מבנה שאומר "הגדר כמה דברים, ואז השתמש בהם בביטוי אחד". כלומר בבלוק שבין let to in אנחנו מגדירים – ובבלוק האחרון בונים ביטוי שמשתמש בהגדרות שביצענו בשלב שלפני.

שאלה הבאה – מדוע שנרצה פונקציית עזר? "סימטרי" היא תכונה של עץ אחד. אבל קשה לבדוק אותה ישירות באופן רקורסיבי – אם נבדוק אם כל תת העץ השמאלי סימטרי, זה לא עוזר. תת העץ השמאלי לא צריך להיות סימטרי בעצמו. הוא צריך להיות תמונת מראה של הימני. לכן נגדיר שאלה על שני העצים – האם A הוא תמונת מראה של B באשר A וB הם עצים?

כמו שכבר דיברנו, לכל פונקציה בSML ישנו ארגומנט אחד. לis_mirror הארגומנט הוא TUPLE – זוג של עצים. נזכר כי עץ הוא EMPTY או Node(val,left,right). בשלב הראשון: אם שני העצים ריקים – אזי נחזיר TRUE. אכן מראה זה של זה.

אחרת, שני העצים לא ריקים. התבנית למעשה מפרקת כל אחד מהם:

- Node(_,l1,r1) הוא העץ הראשון – l1 מקבל את תת העץ השמאלי שלו וr1 את תת העץ הימני שלו.
- Node(_,l2,r2) – אותו הדבר, לעץ השני.

ה"_" פירושו יש כאן משהו, לא מעניין אותי מדוע. המטרה היא לבדוק מבנה – ולא שוויון ערכים ספציפיים.

ואז הבדיקה היא כזו: שני עצים הם תמונת מראה אחד של השני, אם השמאלי של העץ הראשון הוא מראה של הימני של השני וגם הימני של הראשון הוא מראה של השמאלי של השני. אחרת – מוגדר להיות שקר.

החלק האחרון: שם קובעים שעץ ריק הוא סימטרי, ואחרת שולחים את בניו לפונקציית עזר.